DECISION TREE

A **Decision Tree** is a **supervised machine learning algorithm** used for **Classification** and **Regression**.

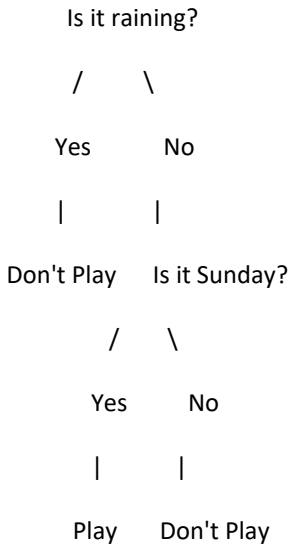
It works exactly like the way humans make decisions by asking **Yes/No questions**.

Real-Life Example

Imagine your friend asks:

"Should I play cricket today?"

You think like this:



This is exactly how a **Decision Tree** works.

It asks one question at a time until it reaches the final decision.

WHAT DOES A DECISION TREE DO?

It learns from the training data and creates a tree of questions.

Example:
Student

Study Hours > 4?

Yes
|
Attendance > 75?

Yes
|
Pass

No
|
Fail

No
|
Fail

Every question divides the data into smaller groups.

WHEN SHOULD WE USE DECISION TREE?

Use it when you want to predict

- Pass / Fail
- Yes / No
- Spam / Not Spam
- Disease / No Disease
- Loan Approved / Rejected
- Customer Churn
- Employee Leave

It can also predict numbers like house prices (regression).

WHY IS IT CALLED A TREE?

Because it looks like an upside-down tree.

Root Node

|

| |

Yes Branch No Branch

| |

Decision Decision

| |

Leaf Node Leaf Node

IMPORTANT TERMS

1. Root Node

The first question.

Example

Study Hours > 4?

---

2. Branch

Possible answers.

Yes

No

---

3. Internal Node

Another question after the first one.

Example

Attendance > 75?

---

4. Leaf Node

Final answer.

Pass

Fail

EXAMPLE DATASET

Study Hours	Attendance	Result
2	60	Fail
3	70	Fail

Study Hours	Attendance	Result
4	80	Pass
5	90	Pass
6	95	Pass
1	50	Fail

How Decision Tree Learns

Suppose this is the data.

Study Hours	Result
1	Fail
2	Fail
3	Fail
4	Pass
5	Pass
6	Pass

The algorithm finds the best question.

For example

Study Hours > 3.5 ?

If Yes

Pass

If No

Fail

The model has learned the rule.

Python Example 1 (Student Pass Prediction)

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = {
   "StudyHours":[1,2,3,4,5,6],
   "Result":["Fail","Fail","Fail","Pass","Pass","Pass"]
}

df = pd.DataFrame(data)

df["Result"] = df["Result"].map({
   "Fail":0,
   "Pass":1
})

X = df[["StudyHours"]]
y = df["Result"]

model = DecisionTreeClassifier()

model.fit(X,y)

prediction = model.predict([[5]])

if prediction[0]==1:
   print("Pass")
else:
   print("Fail")
```

Output

Pass

ADVANTAGES

- Easy to understand and explain.
- Can handle both numerical and categorical data.
- No need for feature scaling.
- Can solve both classification and regression problems.
- Easy to visualize.

LIMITATIONS

- Can overfit the training data if the tree becomes very deep.
- Small changes in the data can produce a different tree.
- A single decision tree may not generalize as well as ensemble methods like Random Forest.

DECISION TREE VS LOGISTIC REGRESSION

Logistic Regression	**Decision Tree**
Uses a mathematical equation to separate classes	Uses a series of Yes/No questions
Works best when the relationship is approximately linear	Can learn linear and non-linear relationships
Predicts probabilities and classes	Predicts by following decision rules
Sensitive to feature scaling in some cases	Does not require feature scaling
Simple and fast	Easy to interpret visually